ĐẶC TẢ THIẾT KẾ gemm_vpu

Module của `gemm_accel_top` · instance `u_vpu`

Mục lục

1 Tổng quan **2**

1.1 Tính năng chính . . . 2

1.2 Tham số . . . 3

1.3 Tín hiệu giao tiếp . . . 4

2 Đặc tả thanh ghi **6**

3 Mô tả chức năng **7**

3.1 Tổ chức chức năng . . . 7

3.2 Module con . . . 7

3.3 Timing . . . 10

3.4 Reset và soft clear . . . 12

3.5 Quy tắc hiện thực . . . 13

1 Tổng quan

1.1 Tính năng chính

VPU nằm giữa đường xả của systolic array và cổng ghi buffer C. Mỗi chu kỳ nó nhận tối đa một đoạn hàng 8 giá trị INT32 từ array. Khi POST_EN = 1, nó cộng bias, requantize, ReLU và kẹp từng lane, rồi ghi Y (INT8) vào buffer C. Khi POST_EN = 0, nó ghi nguyên C (INT32). Độ trễ cố định LAT = 4 chu kỳ, thông lượng một đoạn hàng mỗi chu kỳ, không bao giờ chờ và không có đầu ra lỗi. Trong tài liệu này POST_EN và RELU_EN là cấu hình của job, tương ứng các port i_post_en và i_relu_en; không có tham số module nào mang tên đó.

Chức năng

- Với mỗi đoạn hàng (i, j_0, mask), lane c hợp lệ ứng với j = $j_0 + c$. Khi POST_EN = 1, ghi Y theo công thức dưới đây ở dạng INT8; khi POST_EN = 0, ghi nguyên C ở dạng INT32
- Lane có mask = 0 không được ghi. Mỗi đoạn hàng vào sinh đúng một lệnh ghi ra, đúng LAT = 4 chu kỳ sau
- Kết quả khớp từng bit với công thức chuẩn dưới đây (mô hình tham chiếu post_ref() của gemm_vpu_model.py; mô hình nằm ở gemm-accel/model/, đã chạy lại với baseline ngày 12/11/2026: requant khớp 203 136 trường hợp, pipeline 600 test, 0 lỗi) với mọi giá trị đầu vào INT32, mọi bias INT32, mọi M_0 INT32, mọi sh 0 … 31
- VPU chỉ đọc buffer BIAS và chỉ ghi buffer C; không đọc buffer C, không đọc A, B, không truy cập DRAM, không giữ trạng thái giữa các tác vụ
- VPU không có đầu ra lỗi
- Tài nguyên: 8 lane = 8 bộ nhân 32×32, 8 bộ cộng bias, 8 bộ requant, khoảng 32 DSP trên FPGA; mục tiêu tần số 100 MHz

$$
\begin{aligned}
acc &= C[i][j] + bias[j] && \text{(INT32, wrap mod } 2^{32}\text{)} \\
prod &= acc \cdot M_0 && \text{(tích có dấu 64 bit, không bao giờ tràn)} \\
y &= (prod + 2^{30+sh}) \ggg (31+sh) && \text{(dịch số học: làm tròn nửa lên)} \\
y &= \max(y, 0) \text{ nếu RELU_EN = 1} \\
Y[i][j] &= \min(\max(y, -128), 127) && \text{(INT8)}
\end{aligned}
$$

Bias là độ lệch có dấu dùng chung theo cột. Tích với M0 và dịch 31 + sh ánh xạ giá trị tích lũy sang miền INT8; ReLU (khi RELU_EN = 1) chặn giá trị âm trước khi kẹp.

1.1.1 Dấu, mở rộng dấu và tràn số

> **QUYẾT ĐỊNH BASELINE — D-27**
> Bảng và các quy tắc dưới đây là văn bản chuẩn về signedness, mở rộng dấu, wrap và saturation của toàn bộ đường dữ liệu (array và VPU). Mọi tài liệu khác dẫn lại phải khớp từng chữ.

Bảng 1.1: Kiểu dữ liệu và quy tắc số học

Đại lượng	Kiểu	Quy tắc
A, B	INT8 có dấu	Bù 2, −128 … 127
Tích $a \cdot b$	16 bit có dấu	Mở rộng dấu lên 32 bit trước khi cộng
C (tích lũy)	INT32 có dấu	Cộng dồn **wrap mod** 2^{32}, không bao giờ saturate. Trên miền hợp lệ $\lvert a \cdot b \rvert \le 2^{14}$ và K ≤ 32 nên $\lvert C \rvert \le 2^{19}$: không bao giờ tràn; quy tắc wrap chỉ để định nghĩa hành vi hình thức
bias	INT32 có dấu	$acc = C + bias[j]$ tính trên INT32, **wrap mod** 2^{32}
QM0 (M_0)	INT32 có dấu	Mọi giá trị 32 bit đều hợp lệ
QSHIFT (sh)	5 bit không dấu	0 … 31

Bảng 1.1: Kiểu dữ liệu và quy tắc số học (tiếp theo)

Đại lượng	Kiểu	Quy tắc
$prod = acc \cdot M_0$	64 bit có dấu	Không tràn: $\|prod\| \le 2^{62}$
y	33 bit có dấu (hiện thực)	$y = (prod + 2^{30+sh}) \ggg (31 + sh)$: dịch **số học**, làm tròn **nửa lên** (về phía +∞). $2^{30+sh} \le 2^{61}$ và $\|prod\| \le 2^{62}$ nên phép cộng không tràn 64 bit
ReLU	—	Khi `RELU_EN` = 1: $y = \max(y, 0)$
Y	INT8 có dấu	$Y = \min(\max(y, -128), 127)$ — **chỗ duy nhất trong toàn thiết kế có saturation**; mọi chỗ khác là wrap

- **Hiện thực hai bước** (tương đương biểu thức chuẩn): $t = prod \ggg (30 + sh)$ giữ **34 bit**, $y = (t + 1) \ggg 1$ ở **33 bit**; rồi kẹp: $y > 127 \to 127$; $lo = (\texttt{RELU_EN}\ ?\ 0 : -128)$; $y < lo \to lo$; còn lại $y[7:0]$
- **Bắt buộc giữ đủ 34/33 bit**: cắt t về 32 bit cho kết quả sai tại $acc = M_0 = -2^{31}$. Khi đó $prod = 2^{62}$, $sh = 0$: $t = 2^{32}$ (vừa trong 34 bit), $y = 2^{31}$, kẹp được $Y = 127$; nếu cắt t về 32 bit thì $t = 0$, $y = 0$ và $Y = 0$ — sai
- Khi `POST_EN` = 0, VPU đi thẳng: `o_y` = $prod3[31:0]$ (chính là C INT32) và `o_c_wr_fmt` = 0

1.2 Tham số

Bảng 1.2: Tham số

Tham số	Giá trị	Ý nghĩa
`P`	8	Số lane, cố định. Nối từ `PE_ARRAY_SIZE` của top, cùng giá trị với array
`MUL_STAGES`	1	Số tầng của bộ nhân $acc \cdot M_0$. Nối từ `gemm_pkg::VPU_MUL_STAGES` = 1
`LAT`	4	localparam = 3 + `MUL_STAGES`: độ trễ từ `i_row_valid` tới `o_c_wr_en`
`IDX_W`	5	localparam: độ rộng chỉ số hàng, cột (0 … 31)

Bảng 1.3: Hằng dẫn xuất từ P (không phải tham số)

Hằng	Giá trị	Công thức, nơi dùng
`ROW_VEC_WIDTH`	256 bit	32·P: độ rộng `i_row_data`, `i_bias_rd_data`, `o_c_wr_data`
`AB_VEC_WIDTH`	64 bit	8·P: không có port nào của VPU dùng; liệt kê để thống nhất với array
Q, LP	2, 3	P/4 và $\log_2 P$: dùng ở SRAM; VPU chỉ dùng LP để kiểm `i_row_col`[2:0] = 0

Kiểm tra lúc elaboration

- P ≠ 8 hoặc `MUL_STAGES` ≠ 1 phải bị chặn lúc elaboration (`$error` trong `generate`)
- Độ trễ chỉ khai báo một lần, trong package `gemm_pkg`: `VPU_MUL_STAGES` = 1 và `VPU_LATENCY` = 3 + `VPU_MUL_STAGES` = 4. `gemm_accel_top` nối `MUL_STAGES` = `gemm_pkg::VPU_MUL_STAGES` cho VPU và `L_VPU` = `gemm_pkg::VPU_LATENCY` cho array. Không được viết số 4 trực tiếp ở top
- `gemm_vpu` có kiểm tra elaboration: `LAT == gemm_pkg::VPU_LATENCY`; sai thì `$error`. Array kiểm tra đối xứng `L_VPU == gemm_pkg::VPU_LATENCY`

> **QUYẾT ĐỊNH BASELINE — D-05, D-20**
> `MUL_STAGES` = 1, `LAT` = 4, `L_VPU` = 4. Ở 100 MHz (10 ns) một tầng nhân 32×32 → 64 bit là đủ thời gian. Nếu tổng hợp không đạt 100 MHz, cách sửa đã định trước là đổi `gemm_pkg::VPU_MUL_STAGES` thành 2 (`LAT` = 5, `L_VPU` = 5); đó là một lần phát hành lại baseline, không phải chế độ chạy của baseline này.

1.3 Tín hiệu giao tiếp

Bảng 1.4: Tín hiệu giao tiếp của `gemm_vpu`

Tên tín hiệu	Độ rộng	Hướng	Mô tả
Clock, reset			
`clk`	1	Input	Clock của khối. Các thanh ghi cập nhật tại cạnh lên.
`rst_n`	1	Input	Reset tích cực mức thấp; được kích hoạt bất đồng bộ. Top cấp `rst_n_sync`: cạnh nhả reset đã được đồng bộ với clock.
`i_soft_clr`	1	Input	Xung xóa đồng bộ dài một chu kỳ do Main FSM phát. Xóa trạng thái điều khiển tại cạnh lên; phạm vi xóa dữ liệu được quy định trong mục Reset.
Cấu hình của job đang tính (từ FSM, `o_comp_*`; ổn định suốt COMPUTE)			
`i_post_en`	1	Input	Cho phép hậu xử lý: 0 xuất INT32 nguyên gốc; 1 cộng bias, lượng tử hóa và xuất INT8. Cấu hình giữ ổn định trong suốt COMPUTE, tới khi pipeline đã ghi xong kết quả cuối.
`i_relu_en`	1	Input	Cho phép chặn kết quả âm về 0 trước khi kẹp INT8. Bị bỏ qua khi hậu xử lý tắt. Cấu hình giữ ổn định trong suốt COMPUTE, tới khi pipeline đã ghi xong kết quả cuối.
`i_quant_m0`	32	Input	Hệ số nhân lượng tử M0, số nguyên 32 bit có dấu (mọi giá trị đều hợp lệ). Cấu hình giữ ổn định trong suốt COMPUTE, tới khi pipeline đã ghi xong kết quả cuối.
`i_quant_shift`	5	Input	Lượng dịch bổ sung sh, số không dấu từ 0 đến 31. Tổng lượng dịch trong công thức lượng tử là 31 + sh. Cấu hình giữ ổn định trong suốt COMPUTE, tới khi pipeline đã ghi xong kết quả cuối.
Luồng hàng vào, từ array (không có ready)			
`i_row_valid`	1	Input	Báo có một đoạn hàng kết quả hợp lệ trong chu kỳ này. Bên nhận phải lấy ngay; giao tiếp không có ready.
`i_row_idx`	5	Input	Chỉ số hàng i của đoạn kết quả. Hợp lệ cùng i_row_valid và phải nhỏ hơn M.
`i_row_col`	5	Input	Cột đầu j0 của đoạn kết quả. Là bội của 8 (bit [2:0] = 0), nhỏ hơn N; hợp lệ cùng i_row_valid.
`i_row_data`	256	Input	Vector 8 giá trị INT32 có dấu (`ROW_VEC_WIDTH`). Lane c tại bit [32c +: 32] chứa C[i][j0+c]; hợp lệ cùng i_row_valid, chỉ dùng lane có mask = 1.
`i_row_mask`	8	Input	Mặt nạ các lane hợp lệ của đoạn hàng: các bit 1 liên tiếp từ bit 0, khác 0. Bit c = 1 khi j0 + c < N; bit c = 0 ngăn ghi phần tử tương ứng. Hợp lệ cùng i_row_valid.
Cổng đọc buffer BIAS (VPU là master, không có ready)			
`o_bias_rd_en`	1	Output	Yêu cầu đọc 8 phần tử bias. Bằng `i_row_valid` ∧ `i_post_en`, tổ hợp trong chính chu kỳ D; dữ liệu về sau một chu kỳ.

Bảng 1.4: Tín hiệu giao tiếp của `gemm_vpu` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`o_bias_rd_col`	5	Output	Cột đầu j0 của vector bias, bằng `i_row_col` (tổ hợp). Là bội của 8; hợp lệ cùng bias read enable.
`i_bias_rd_data`	256	Input	Vector 8 giá trị bias INT32. Lane c tại bit [32c +: 32] chứa bias[j0+c]. Hợp lệ đúng một chu kỳ sau yêu cầu; lane bị mask mang giá trị bất kỳ.
Cổng ghi buffer C (VPU là master, không có ready)			
`o_c_wr_en`	1	Output	Cho phép ghi một đoạn hàng kết quả vào buffer C tại cạnh lên. Các trường fmt, row, col, data và mask hợp lệ trong chu kỳ này.
`o_c_wr_fmt`	1	Output	Chọn định dạng phần tử ghi C. Giá trị 0: INT32, mỗi phần tử là một word. Giá trị 1: INT8, mỗi phần tử là một byte. Hợp lệ cùng write enable; bằng `i_post_en`.
`o_c_wr_row`	5	Output	Chỉ số hàng i của buffer C cần ghi. Hợp lệ cùng write enable.
`o_c_wr_col`	5	Output	Cột đầu j0 của đoạn hàng cần ghi. Là bội của 8; hợp lệ cùng write enable.
`o_c_wr_data`	256	Output	Dữ liệu ghi đoạn hàng C (`ROW_VEC_WIDTH`). Với fmt = 0, phần tử c nằm tại bit [32c +: 32]. Với fmt = 1, phần tử c nằm tại bit [8c +: 8] và bit [255:64] bằng 0. Hợp lệ cùng write enable.
`o_c_wr_mask`	8	Output	Cho phép ghi theo phần tử, bằng `i_row_mask` của đoạn hàng tương ứng. Bit c = 1 ghi phần tử c theo định dạng fmt; bit c = 0 giữ nguyên phần tử đó. Hợp lệ cùng write enable.

1.3.1 Quy tắc giao tiếp

Array → VPU

TRAO ĐỔI GÌ?
Một đoạn hàng 8 giá trị INT32 kèm hàng, cột đầu và mask.

AI KHỞI TẠO?
Array đẩy; VPU nhận. Không có ready.

KHI NÀO ĐƯỢC CHẤP NHẬN?
VPU phải nhận đoạn hàng ở đúng chu kỳ `i_row_valid` = 1, kể cả khi các chu kỳ liền nhau đều có đoạn hàng.

KHI NÀO SANG BƯỚC TIẾP?
VPU ghi đoạn hàng đó vào buffer C đúng `LAT` = 4 chu kỳ sau. Array tính `o_pe_done` dựa trên con số này (`L_VPU` = `LAT`).

VPU ↔ SRAM (BIAS đọc, C ghi)

TRAO ĐỔI GÌ?
Đọc 8 word bias liền nhau; ghi một đoạn hàng của C, có mask.

AI KHỞI TẠO?
VPU là master trên cả hai cổng. SRAM không có backpressure.

KHI NÀO ĐƯỢC CHẤP NHẬN?

Đọc: dữ liệu về đúng 1 chu kỳ sau yêu cầu. Ghi: có hiệu lực ở cạnh clock cuối chu kỳ `o_c_wr_en` = 1.

KHI NÀO SANG BƯỚC TIẾP?

Trong pha COMPUTE: VPU là bên duy nhất đọc BIAS và ghi C.

Giao tiếp

- Nguồn phát đăng ký toàn bộ `i_row_*`. Mỗi cặp (hàng, cột đầu) hợp lệ xuất hiện đúng một lần trong tác vụ. `i_row_col` là bội của 8; mask khác 0, gồm các bit 1 liên tiếp từ bit 0. Lane c hợp lệ khi cột đầu + c < N. VPU nhận dữ liệu ngay chu kỳ valid, không có ready
- `o_bias_rd_en` = `i_row_valid` ∧ `i_post_en`; `o_bias_rd_col` = `i_row_col`. Đây là **đường tổ hợp duy nhất từ input tới output** của VPU (cả thiết kế còn hai đường tổ hợp ở DMA, xem tài liệu hệ thống mục Quy tắc tích hợp), được phép vì `i_row_*` lấy từ thanh ghi của array. Khi `i_post_en` = 0, VPU không bao giờ đọc BIAS (buffer BIAS không được nạp ở tác vụ đó)
- VPU chỉ dùng `i_bias_rd_data` ở chu kỳ ngay sau chu kỳ có `o_bias_rd_en` = 1. Lane có mask = 0 được phép mang giá trị bất kỳ, kể cả X trong mô phỏng; X không được lan sang lane có mask = 1
- Mọi port `o_c_wr_*` lấy từ thanh ghi. Khi `o_c_wr_en` = 1: `o_c_wr_fmt` = `i_post_en`; `o_c_wr_mask` = `i_row_mask` của đoạn hàng, khác 0. Khi `o_c_wr_en` = 0, các port khác không có nghĩa
- Cấu hình `i_post_en`, `i_relu_en`, `i_quant_m0`, `i_quant_shift` ổn định từ `pe_start` tới `pe_done`. Vì pipeline rỗng trước đoạn hàng đầu và sau lệnh ghi cuối của mỗi tác vụ, VPU dùng cấu hình trực tiếp, không chốt hay đưa cấu hình theo pipeline
- Hàng và cột thừa của tile biên (M, N không chia hết cho 8) không tới VPU dưới dạng lane hợp lệ: hàng thừa không được array xả, cột thừa bị chặn bởi mask (D-26); VPU không cần biết M, N

1.3.2 Ngữ nghĩa lệnh ghi C

Giao tiếp ghi C có ngữ nghĩa như bảng dưới; mỗi bit mask điều khiển một phần tử, với kích thước phần tử do fmt quyết định.

Bảng 1.5: Mỗi lane c có `o_c_wr_mask[c]` = 1, j = j0 + c

`o_c_wr_fmt`	Word được ghi	Nội dung
0 (INT32)	{i[4:0], j[4:0]}	Cả word ← `o_c_wr_data[32c+31 : 32c]`
1 (INT8)	{i[4:0], j[4:2]}	Byte lane j[1:0] ← `o_c_wr_data[8c+7 : 8c]`; ba byte còn lại của word giữ nguyên

Với P = 8, một đoạn hàng INT8 là hai word liền nhau {i, $j_0/4$} và {i, $j_0/4+1$} (lane 0 … 3 vào word thứ nhất, lane 4 … 7 vào word thứ hai). Vì N là bội của 4, mask ở chế độ INT8 luôn phủ trọn word, nhưng SRAM vẫn dùng byte enable theo mask.

2 Đặc tả thanh ghi

Module không có thanh ghi ánh xạ MMIO. Cấu hình (`i_post_en`, `i_relu_en`, `i_quant_m0`, `i_quant_shift`) được nhận qua các tín hiệu đầu vào từ Main FSM, bản gốc nằm trong descriptor (`FLAGS`, `QM0`, `QSHIFT`); các thanh ghi MMIO của hệ thống nằm trong `gemm_register_file`, xem đặc tả hệ thống baseline. Trạng thái nội bộ, điều kiện chốt và reset được mô tả bên dưới.

3 Mô tả chức năng

3.1 Tổ chức chức năng

VPU là pipeline thẳng LAT = 4 tầng V1 … V4, không có trạng thái điều khiển: một chuỗi bit hợp lệ và metadata (hàng, cột, mask) chạy song song với 8 lane dữ liệu giống hệt nhau. Mỗi lane là một gemm_vpu_lane. Chế độ POST_EN = 0 đi qua cùng các tầng, nên độ trễ không đổi theo chế độ.

Bảng 3.1: Các tầng pipeline (D = chu kỳ có i_row_valid = 1)

Tầng	Thanh ghi	Tính trong chu kỳ trước khi chốt	Có giá trị ở chu kỳ
V1	c1, meta1, vld1	Chốt i_row_data, i_row_idx/col/mask; phát đọc bias (tổ hợp ở D)	D + 1
V2	acc2, meta2, vld2	acc2 = post ? c1 + bias : c1 (32 bit wrap)	D + 2
V3	prod3, meta3, vld3	prod3 = post ? acc2 × M0 : sext(acc2) (64 bit), một tầng thanh ghi	D + 3
V4	o_c_wr_*	requant(prod3) nếu post = 1, prod3[31:0] nếu post = 0; đóng gói theo fmt	D + 4

Bảng 3.2: Module con

Module	Instance	Vai trò
gemm_vpu_lane	u_lane[c]	Đường dữ liệu một lane (c = 0 … 7): thanh ghi c1, acc2, prod3; bộ cộng bias; bộ nhân 32 × 32; đầu ra tổ hợp o_y
gemm_vpu_requant	u_rq (trong lane)	Tổ hợp: dịch có làm tròn, ReLU, kẹp; 64 bit → INT8
(logic trong top)	—	Chuỗi bit hợp lệ và metadata V1 … V3; cổng đọc bias; thanh ghi đầu ra V4 và đóng gói

Bảng 3.3: Kiểm kê thanh ghi (P = 8, MUL_STAGES = 1)

Thanh ghi	Số bit	Loại
c1 (8 lane × 32 bit)	256	Dữ liệu, không reset
acc2 (8 lane × 32 bit)	256	Dữ liệu, không reset
prod3 (8 lane × 64 bit)	512	Dữ liệu, không reset
Tổng thanh ghi dữ liệu lane	1 024	Dữ liệu, không reset
o_c_wr_data	256	Dữ liệu, không reset
o_c_wr_fmt, o_c_wr_row, o_c_wr_col, o_c_wr_mask	1 + 5 + 5 + 8 = 19	Dữ liệu, không reset
meta1, meta2, meta3 (mỗi tầng idx 5 + col 5 + mask 8 = 18 bit)	54	Dữ liệu, không reset
vld1, vld2, vld3, o_c_wr_en	4	Trạng thái, reset bất đồng bộ về 0
Tổng đầu ra, metadata và bit hợp lệ	333	256 + 19 + 54 + 4

3.2 Module con

3.2.1 gemm_vpu_lane

Bảng 3.4: Tín hiệu giao tiếp của `gemm_vpu_lane`

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`clk`	1	Input	Clock cập nhật các thanh ghi dữ liệu của lane tại cạnh lên. Lane không có reset dữ liệu; tính hợp lệ do module cha quản lý.
`i_post_en`	1	Input	Cho phép hậu xử lý: 0 xuất INT32 nguyên gốc; 1 cộng bias, lượng tử hóa và xuất INT8. Cấu hình giữ ổn định trong suốt COMPUTE, tới khi pipeline đã ghi xong kết quả cuối.
`i_relu_en`	1	Input	Cho phép chặn kết quả âm về 0 trước khi kẹp INT8. Bị bỏ qua khi hậu xử lý tắt. Cấu hình giữ ổn định trong suốt COMPUTE, tới khi pipeline đã ghi xong kết quả cuối.
`i_m0`	32	Input	Hệ số nhân lượng tử M0, INT32 có dấu (khai báo `signed`, hoặc ép `signed'()` trước khi nhân). Giữ ổn định suốt quá trình xử lý các đoạn hàng của job.
`i_sh`	5	Input	Lượng dịch bổ sung sh, không dấu, từ 0 đến 31. Công thức lượng tử dùng tổng lượng dịch 31 + sh.
`i_c`	32	Input	Giá trị tích lũy INT32 có dấu của một lane. Được chốt vào tầng V1 tại cạnh lên.
`i_bias`	32	Input	Giá trị bias INT32 có dấu của cột tương ứng. Được dùng ở tầng V2, cùng chu kỳ với c1 của đoạn hàng đó.
`o_y`	32	Output	Kết quả tổ hợp từ prod3. Khi hậu xử lý bật, mở rộng dấu giá trị INT8 lên 32 bit; khi tắt, lấy prod3[31:0].

`gemm_vpu_lane`

- `c1 <= i_c` mỗi chu kỳ. `acc2 <= i_post_en ? c1 + i_bias : c1`, cộng 32 bit wrap mod 2^{32}. Mux chọn `c1` (không phải cộng với 0) để X của `i_bias` không lan khi không đọc bias
- `prod3` = `acc2` $\times$ `i_m0`, nhân có dấu 32 $\times$ 32 $\rightarrow$ 64 bit, khi `i_post_en` = 1; = `acc2` mở rộng dấu lên 64 bit khi `i_post_en` = 0. Mux đặt ở đầu vào thanh ghi `prod3`, sau bộ nhân tổ hợp
- `prod3` là một thanh ghi sau bộ nhân tổ hợp (`MUL_STAGES` = 1). Lane có đúng 3 tầng thanh ghi từ `i_c` tới `prod3`: `c1`, `acc2`, `prod3`
- `o_y` = `i_post_en` ? {{24{Y[7]}}, Y} : `prod3[31:0]`, với Y = `o_y` của `gemm_vpu_requant(prod3, i_sh, i_relu_en)`. Không có thanh ghi sau `prod3` trong lane
- Thanh ghi dữ liệu của lane cập nhật mọi chu kỳ, không có enable và không có giá trị reset

```
// Tham khảo (SystemVerilog, MUL_STAGES = 1)
logic signed [31:0] c1, acc2;
logic signed [63:0] prod3;
logic signed [7:0]  y8;
always_ff @(posedge clk) begin
  c1    <= i_c;
  acc2  <= i_post_en ? (c1 + i_bias) : c1;
  // hai toán hạng 32 bit có dấu, mở rộng dấu lên 64 bit: tổng hợp ra bộ nhân 32×32 → 64
  prod3 <= i_post_en ? (64'(acc2) * 64'(signed'(i_m0))) : 64'(acc2);
end
gemm_vpu_requant u_rq (.i_prod(prod3), .i_sh(i_sh), .i_relu(i_relu_en), .o_y(y8));
assign o_y = i_post_en ? 32'(y8) : prod3[31:0];   // 32'(y8): mở rộng dấu
```

3.2.2 `gemm_vpu_requant`

Bảng 3.5: Tín hiệu giao tiếp của `gemm_vpu_requant`

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`i_prod`	64	Input	Tích có dấu 64 bit đầu vào của khối requant. Kết quả tổ hợp được tính theo giá trị này, sh và cờ ReLU.
`i_sh`	5	Input	Lượng dịch bổ sung sh, không dấu, từ 0 đến 31. Công thức lượng tử dùng tổng lượng dịch 31 + sh.
`i_relu`	1	Input	Cho phép ReLU trong khối requant. Khi bằng 1, chặn giá trị âm về 0 trước khi kẹp INT8.
`o_y`	8	Output	Kết quả INT8 có dấu sau làm tròn, ReLU (nếu bật) và kẹp. Đây là đầu ra thuần tổ hợp.

Công thức chuẩn $y = (prod + 2^{30+sh}) \ggg (31 + sh)$ cần cộng 64 bit rồi dịch 64 bit. Module hiện thực đúng một dạng tương đương rẻ hơn:

$$t = prod \ggg (30 + sh) \quad \text{(34 bit)}, \qquad y = (t + 1) \ggg 1 \quad \text{(33 bit)}$$

$$Y = \begin{cases} 127 & y > 127 \\ lo & y < lo \\ y[7:0] & \text{còn lại} \end{cases}, \qquad lo = \begin{cases} 0 & relu = 1 \\ -128 & relu = 0 \end{cases}$$

> **VÌ SAO HAI CÁCH CHO CÙNG KẾT QUẢ**
> Đặt $n = 31 + sh$ và viết $prod = t \cdot 2^{n-1} + r$ với $0 \le r < 2^{n-1}$ (tức $t = prod \ggg (n-1)$). Khi đó $(prod + 2^{n-1})/2^n = (t+1)/2 + r/2^n$, với $0 \le r/2^n < 1/2$. Nếu $t+1$ chẵn, phần nguyên là $(t+1)/2$; nếu $t+1$ lẻ, $(t+1)/2 = q + 1/2$ và cộng thêm một số nhỏ hơn 1/2 vẫn cho phần nguyên q. Cả hai trường hợp đều bằng $(t+1) \ggg 1$. ReLU rồi kẹp tương đương với kẹp vào $[lo, 127]$. Theo `spec_1`, mô hình của nguồn kiểm tra 203 136 trường hợp, gồm mọi tổ hợp biên và các giá trị đúng nửa; mô hình đó không có trong thư mục nguồn và chưa được chạy lại. Phép kiểm chạy riêng khi soạn baseline cho kết quả khớp từng bit trên 800 576 trường hợp (xem tài liệu quyết định, mục Giới hạn của bộ tài liệu).

`gemm_vpu_requant`

- Kết quả phải bằng công thức chuẩn với mọi `i_prod` trong $[-2^{62} + 2^{31}, 2^{62}]$ (miền của $acc \cdot M_0$), mọi `i_sh`, mọi `i_relu`
- Độ rộng: $|prod| \le 2^{62}$ và dịch ≥ 30 nên $t \in [-2^{32}, 2^{32}]$, cần **34 bit có dấu**; y cần **33 bit có dấu**. Không được cắt ngắn hơn: cắt t về 32 bit sai ở $acc = M_0 = -2^{31}$ (mục Dấu, mở rộng dấu và tràn số)
- Lượng dịch $30 + sh \in$ [30, 61]. Bộ dịch chỉ sinh 34 bit của t, tức cửa sổ `i_prod[63 : 30+sh]` mở rộng dấu lên 34 bit, hiện thực bằng mux 32 vị trí, không dùng bộ dịch 64 bit đầy đủ
- Dịch là dịch số học (giữ dấu); làm tròn là nửa lên (về phía +∞), không phải nửa ra xa 0

```
// Tham khảo (SystemVerilog)
logic signed [33:0] t;
logic signed [32:0] y, lo;
assign t  = 34'(signed'(i_prod) >>> (6'd30 + 6'(i_sh)));  // dịch số học
assign y  = 33'((35'(t) + 35'sd1) >>> 1);
assign lo = i_relu ? 33'sd0 : -33'sd128;
always_comb
  if      (y > 33'sd127) o_y = 8'sd127;
  else if (y < lo)       o_y = 8'(lo);
  else                   o_y = 8'(y);
```

3.2.3 Logic trong top

Logic trong `gemm_vpu`

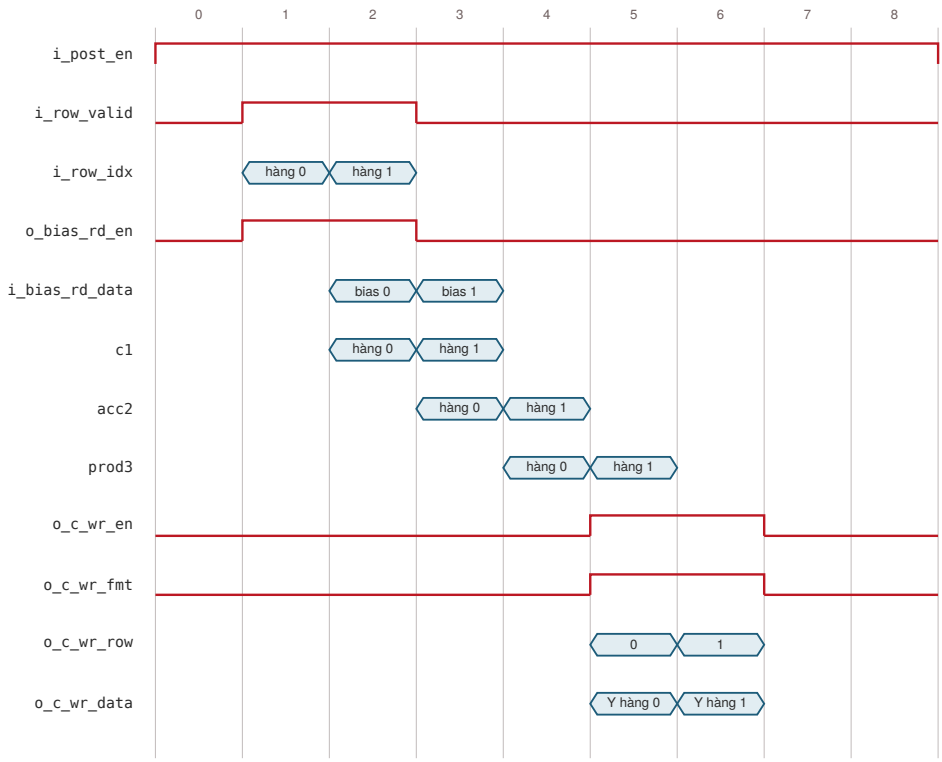
- Chuỗi hợp lệ: `vld1 <= i_row_valid`, `vld2 <= vld1`, `vld3 <= vld2`, `o_c_wr_en <= vld3`. Metadata (idx, col, mask) đi cùng chuỗi đó. Mọi bit hợp lệ reset và clear về 0; metadata không có giá trị reset
- Đọc bias khi row valid và post enable cùng bằng 1, tổ hợp trong chu kỳ D, dùng cột đầu của đoạn hàng. Lane c nhận `i_c = i_row_data[32c +: 32]`, `i_bias = i_bias_rd_data[32c +: 32]`
- Thanh ghi đầu ra V4, nạp mọi chu kỳ: `o_c_wr_fmt <= i_post_en`; `o_c_wr_row`, `o_c_wr_col`, `o_c_wr_mask <=` metadata của `vld3`. Dữ liệu: khi `i_post_en` = 0, `o_c_wr_data[32c +: 32] <= o_y` của lane c; khi `i_post_en` = 1, `o_c_wr_data[8c +: 8] <= o_y[7:0]` của lane c và `o_c_wr_data[255 : 64] <= 0`
- Không có bộ đếm, FSM, hay trạng thái nào khác. VPU không biết M, N, K và không biết khi nào tác vụ kết thúc

3.3 Timing

Mọi lệnh ghi xảy ra đúng LAT = 4 chu kỳ sau đoạn hàng tương ứng. Thông lượng là một đoạn hàng mỗi chu kỳ với mọi mẫu đến (liền nhau hoặc có khoảng trống). Các công thức dưới đây là quy định; RTL phải khớp đúng từng chu kỳ.

Bảng 3.6: Thời điểm các sự kiện (D = chu kỳ có `i_row_valid` = 1)

Sự kiện	Chu kỳ
`o_bias_rd_en`, `o_bias_rd_col` (nếu `POST_EN` = 1; tổ hợp)	D
`i_bias_rd_data` hợp lệ, `c1` hợp lệ	D + 1
`acc2` hợp lệ	D + 2
`prod3` hợp lệ	D + 3
`o_c_wr_en` = 1, dữ liệu ghi vào C ở cạnh clock cuối chu kỳ	D + LAT = D + 4
`o_pe_done` của array (đoạn hàng cuối ở D_{last})	D_{last} + LAT + 1 = D_{last} + 5



Hình 3.1: Pipeline V1 … V4 với `POST_EN` = 1 cho hai đoạn hàng liên tiếp (hàng 0 ở D = 1, hàng 1 ở D = 2, cùng j0 = 0, mask 8'hFF). `o_bias_rd_en` lên cùng chu kỳ `i_row_valid` (đường tổ hợp); bias và `c1` ở D + 1, `acc2` ở D + 2, `prod3` ở D + 3, `o_c_wr_en` ở D + 4 = chu kỳ 5 và 6; `o_c_wr_fmt` = `i_post_en` = 1

QUYẾT ĐỊNH BASELINE — D-06

r_{last} là **chỉ số** của hàng hợp lệ cuối của tile cuối, tức (số hàng hợp lệ của tile cuối) − 1. Thời gian COMPUTE khi tích hợp với array:

$$\text{COMPUTE_CYCLES} = (N_T - 1) \cdot \max(K, P) + K + P + r_{last} + \text{LAT} + 3$$
$$N_T = \lceil M/8 \rceil \lceil N/8 \rceil, \qquad r_{last} = M - 8(\lceil M/8 \rceil - 1) - 1$$

Kiểm chứng M = N = K = 32: $T_M = T_N = 4$, $N_T = 16$, $I = \max(32, 8) = 32$, $r_{last} = 7$, LAT = 4 → 15·32 + 32 + 8 + 7 + 4 + 3 = **534 chu kỳ** = 5,34 µs ở 100 MHz.
Hiệu chỉnh so với spec_1: bản spec_1 của VPU định nghĩa r_{last} là số hàng hợp lệ (1 … P); với định nghĩa đó công thức lệch 1 chu kỳ so với con số 534 (và so với tài liệu array). Baseline dùng định nghĩa chỉ số ở mọi tài liệu.

Bảng 3.7: Số chu kỳ từ `pe_start` tới `pe_done` khi có VPU (P = 8, LAT = 4)

(M, N, K)	Số chu kỳ	Tính
(32, 32, 32)	534	$15 \cdot 32 + 32 + 8 + 7 + 4 + 3$
(20, 12, 8)	66	$N_T = 6$, $r_{last} = 3$: $5 \cdot 8 + 8 + 8 + 3 + 4 + 3$
(8, 8, 4)	26	$0 + 4 + 8 + 7 + 4 + 3$
(2, 8, 4) (ví dụ bên dưới)	20	$r_{last} = 1$: $0 + 4 + 8 + 1 + 4 + 3$

3.3.1 Ví dụ số học và đóng gói

Ví dụ với M = 2, N = 8, K = 4, một tile. Array xả hàng 0 ở chu kỳ 14 và hàng 1 ở chu kỳ 15 ($D_0(r) = 14 + r$); `o_pe_done` ở 15 + 5 = 20. Cấu hình `POST_EN` = 1, `RELU_EN` = 1, M_0 = 1 717 986 918, sh = 5, $bias = (500, -300, 200, -1000, 100, 3000, -50, 25)$. Hàng 0 có $c1 = (-58, 219, 250, 7, 1000, -2000, -1, 0)$; hàng 1 có $c1 = (92, 93, -173, -14, -500, 64, 3, -3)$.

Bảng 3.8: Giá trị từng tầng của ví dụ

(i, j)	c1	bias	acc2	prod3	t = prod ⋙ 35	y	Y
(0,0)	−58	500	442	759 350 217 756	22	11	11
(0,1)	219	−300	−81	−139 156 940 358	−5	−2	0
(0,2)	250	200	450	773 094 113 100	22	11	11
(0,3)	7	−1000	−993	−1 705 961 009 574	−50	−25	0
(0,4)	1000	100	1100	1 889 785 609 800	54	27	27
(0,5)	−2000	3000	1000	1 717 986 918 000	49	25	25
(0,6)	−1	−50	−51	−87 617 332 818	−3	−1	0
(0,7)	0	25	25	42 949 672 950	1	1	1
(1,0)	92	500	592	1 017 048 255 456	29	15	15
(1,1)	93	−300	−207	−355 623 292 026	−11	−5	0
(1,2)	−173	200	27	46 385 646 786	1	1	1
(1,3)	−14	−1000	−1014	−1 742 038 734 852	−51	−25	0
(1,4)	−500	100	−400	−687 194 767 200	−20	−10	0
(1,5)	64	3000	3064	5 263 911 916 752	153	77	77
(1,6)	3	−50	−47	−80 745 385 146	−3	−1	0
(1,7)	−3	25	22	37 795 712 196	1	1	1

Lệnh ghi ở chu kỳ 18 (D = 14) có `o_c_wr_fmt` = 1, hàng 0, cột 0, mask 8'hFF, dữ liệu byte 0 … 7 = (11, 0, 11, 0, 27, 25, 0, 1), tức word {0, 0} của C = `0x000B000B` và word {0, 1} = `0x0100191B`. Chu kỳ 19 ghi hàng 1 với byte 0 … 7 = (15, 0, 1, 0, 0, 77, 0, 1): word {1, 0} = `0x0001000F`, word {1, 1} = `0x01004D00`. Với `POST_EN` = 0, cùng hai chu kỳ đó ghi tám word INT32 mỗi hàng (`o_c_wr_fmt` = 0): (−58, 219, 250, 7, 1000, −2000, −1, 0) và (92, 93, −173, −14, −500, 64, 3, −3), và không có `o_bias_rd_en`.

3.4 Reset và soft clear

> **QUYẾT ĐỊNH BASELINE — D-21**
> Bit hợp lệ dùng `always_ff@(posedgeclkornegedgerst_n)` (`rst_n` ở đây là `rst_n_sync` do top cấp). Thanh ghi dữ liệu thuần (`c1`, `acc2`, `prod3`, metadata, `o_c_wr_fmt/row/col/data/mask`) dùng `always_ff@(posedgeclk)` và không có giá trị reset. Không có latch.

Reset

- `rst_n` = 0 (bất đồng bộ): `vld1`, `vld2`, `vld3`, `o_c_wr_en` về 0
- `i_soft_clr` = 1 (đồng bộ): xóa `vld1`, `vld2`, `vld3` và `o_c_wr_en` về 0 ở cạnh clock cuối chu kỳ đó; mọi đoạn hàng đang trong pipeline bị bỏ. Nếu `o_c_wr_en` đã bằng 1 trong chính chu kỳ có `i_soft_clr`, lệnh ghi đó vẫn xảy ra; điều này chấp nhận được vì nội dung scratchpad sau soft reset là không xác định
- Thanh ghi dữ liệu (`c1`, `acc2`, `prod3`, metadata, `o_c_wr_fmt/row/col/data/mask`) không có giá trị reset; mọi đường dùng chúng đều bị chặn bởi bit hợp lệ

3.5 Quy tắc hiện thực

Hiện thực

- SystemVerilog tổng hợp được; `always_ff@(posedgeclkornegedgerst_n)` cho bit hợp lệ, `always_ff@(posedgeclk)` cho thanh ghi dữ liệu thuần; không có latch
- Mọi phép cộng, nhân, dịch, so sánh dùng kiểu `signed` rõ ràng; dịch dùng `>>>` trên toán hạng có dấu. Không dùng phép chia hay modulo
- Mỗi module con trong file riêng, cùng tên module; 8 lane dựng bằng `generate`
- Chỉ cổng đọc bias (`o_bias_rd_en`, `o_bias_rd_col`) phụ thuộc tổ hợp vào đầu vào; mọi ngõ ra khác của `gemm_vpu` được đăng ký
- Đường dài nhất dự kiến là bộ nhân 32 × 32 (tầng V3) và chuỗi dịch, cộng 34 bit, so sánh (tầng V4). Phép dịch nằm trong V4; `LAT` = 4 là cố định
- Lane có mask = 0 mang giá trị bất kỳ, kể cả X; giá trị đó không được lan sang lane có mask = 1 và không được tới `o_c_wr_data` của lane được ghi (mỗi lane độc lập, mux của `acc2` chọn `c1` khi `i_post_en` = 0)
- VPU không có đầu ra lỗi

Assertion bắt buộc trong RTL (SVA, chỉ trong mô phỏng)

- **AS-01**: `i_row_valid` → `i_row_col[2:0] == 0`
- **AS-02**: `i_row_valid` → (`i_row_mask != 0`) ∧ `$onehot({1'b0, i_row_mask} + 9'd1)` (mask là dãy bit 1 liền nhau từ bit 0, kể cả toàn 1; cộng ở 9 bit để 8'hFF không tràn về 0)
- **AS-03**: `o_bias_rd_en` → `i_post_en`
- **AS-04**: (`vld1` ∨ `vld2` ∨ `vld3` ∨ `o_c_wr_en`) → `$stable` của cả bốn tín hiệu cấu hình (`i_post_en`, `i_relu_en`, `i_quant_m0`, `i_quant_shift`)
- **AS-05**: `o_c_wr_en` → `o_c_wr_fmt == i_post_en`
- **AS-06**: `o_c_wr_en` ↔ `i_row_valid` trễ đúng `LAT` = 4 chu kỳ (khi không có `i_soft_clr` trong khoảng đó)
- **AS-07**: trong `gemm_accel_top`: `u_array.o_pe_done` → không còn bit hợp lệ nào (`vld1`, `vld2`, `vld3`, `o_c_wr_en`) trong pipeline VPU

Bảng 3.9: Tài nguyên ước tính (P = 8, `MUL_STAGES` = 1)

Hạng mục	Số lượng
Bộ nhân 32 × 32 có dấu → 64 bit	8
Bộ cộng 32 bit (bias)	8
Bộ requant (dịch 32 vị trí × 34 bit, cộng 34 bit, 2 so sánh)	8
Thanh ghi dữ liệu lane (32 + 32 + 64 bit mỗi lane)	1 024 bit
Thanh ghi đầu ra, metadata và bit hợp lệ	333 bit
DSP trên FPGA (khoảng 4 khối mỗi bộ nhân 32 × 32)	khoảng 32
Mục tiêu tần số	100 MHz (10 ns)